

Assignment 5

What was implemented:

Helper functions were added for bilinear and trilinear interpolation. Bilinear interpolation is used to sample the vector field for streamlines, while trilinear interpolation is used for path lines to account for time-varying data. Streamline integration is implemented in both forward and backward directions using either Euler or RK2, with stopping conditions for: leaving the domain, near-zero velocity, and reaching a maximum length. After loading one of the datasets (block, tube, or hurricane), the scalar field is rendered as a grayscale background, with optional rainbow and cool-warm colormaps blended via a slider. Yellow glyph arrows are drawn on a regular grid, scaled by local vector magnitude. Streamlines (orange) and path lines (cyan) are created by placing a seed point using the mouse. For hurricane dataset, advancing the time step updates the scalar texture and glyphs; existing streamlines are calculated again for the new slice.

What new was learned:

A streamline is a snapshot, it traces the field direction at a fixed time step. A pathline is what an actual particle would do, it moves through both space and time simultaneously.

GL_DYNAMIC_DRAW - is used because every frame (glyphs) or every new seed(stream-lines/path-lines) we upload fresh vertex data. The difference between GL_STATIC_DRAW and GL_DYNAMIC_DRAW. The first one tells OpenGL that the data will be set once and read multiple times while the second one tells it that it will be frequently modified.

In order to get the rainbow color coding we set intervals for colors and use built in piecewise linear interpolation function in order to get the colors in between the interval points.

position: 0.00 0.25 0.50 0.75 1.00

color: blue cyan green yellow red

These are the exact color points and in between we use the mix function with the custom calculated blend factor. That's how we derive the rainbow. Also this mix function is used in the mixing of the rainbow color coding with the

grayscale in order to get the insights from the rainbow colors and not lose detail via grayscale.

In order to do the backward and forward integration of the streamlines we just make the integration step negative and this enables us to get the full streamline even if we seed the point in any position.

Problems encountered:

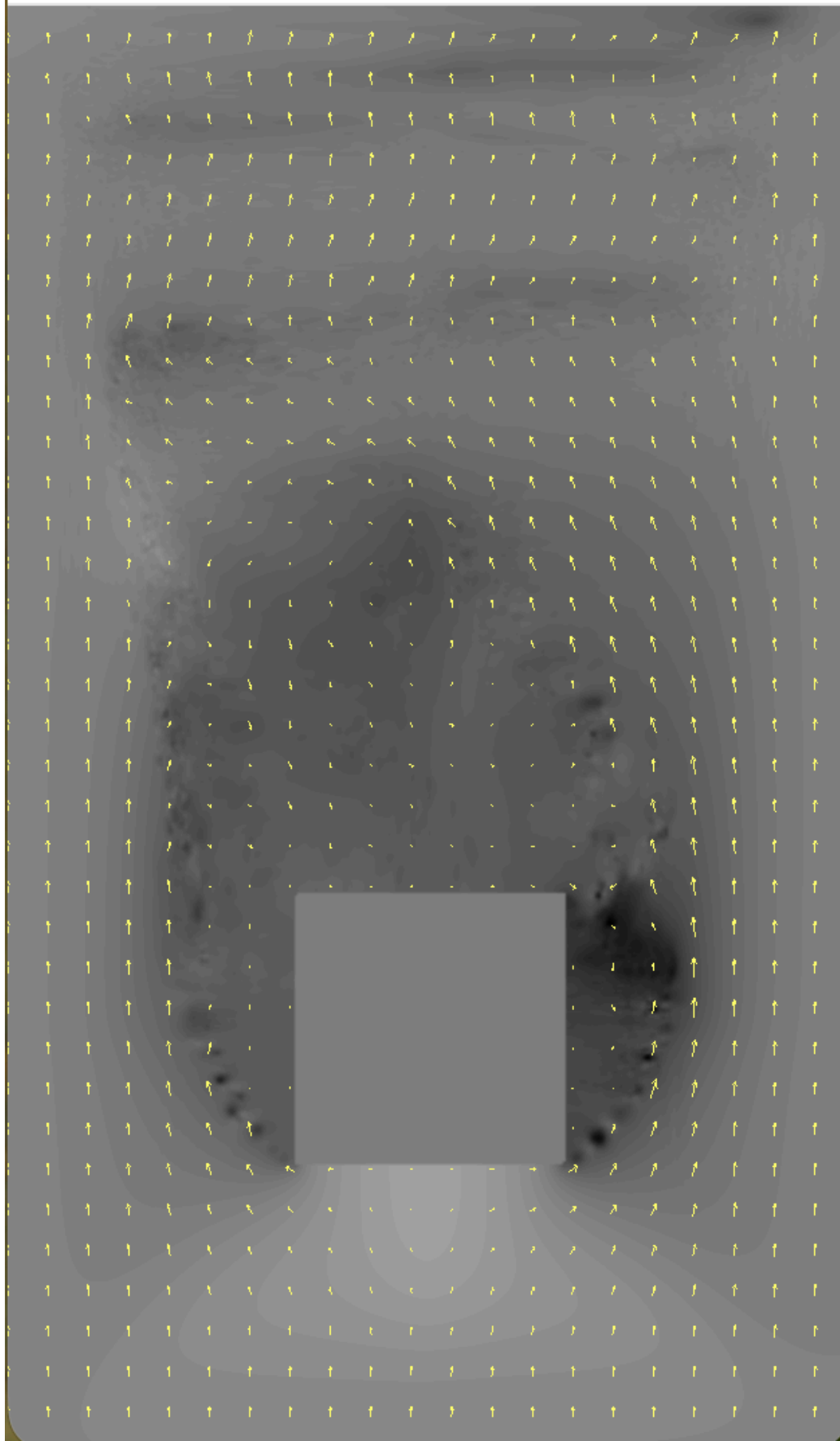
Some path problems were encountered in the code(the paths to the datasets and shaders were incorrect) so I had to fix that first in order to run the code. Also the version of glsl was not compatible with my laptop so I had to change it to 410. Also some enums like GL_INTENSITY16, GL_LUMINANCE and GL_CLAMP caused the screen to appear black since they were already deprecated. After changing them to the new versions the visualization was fixed.

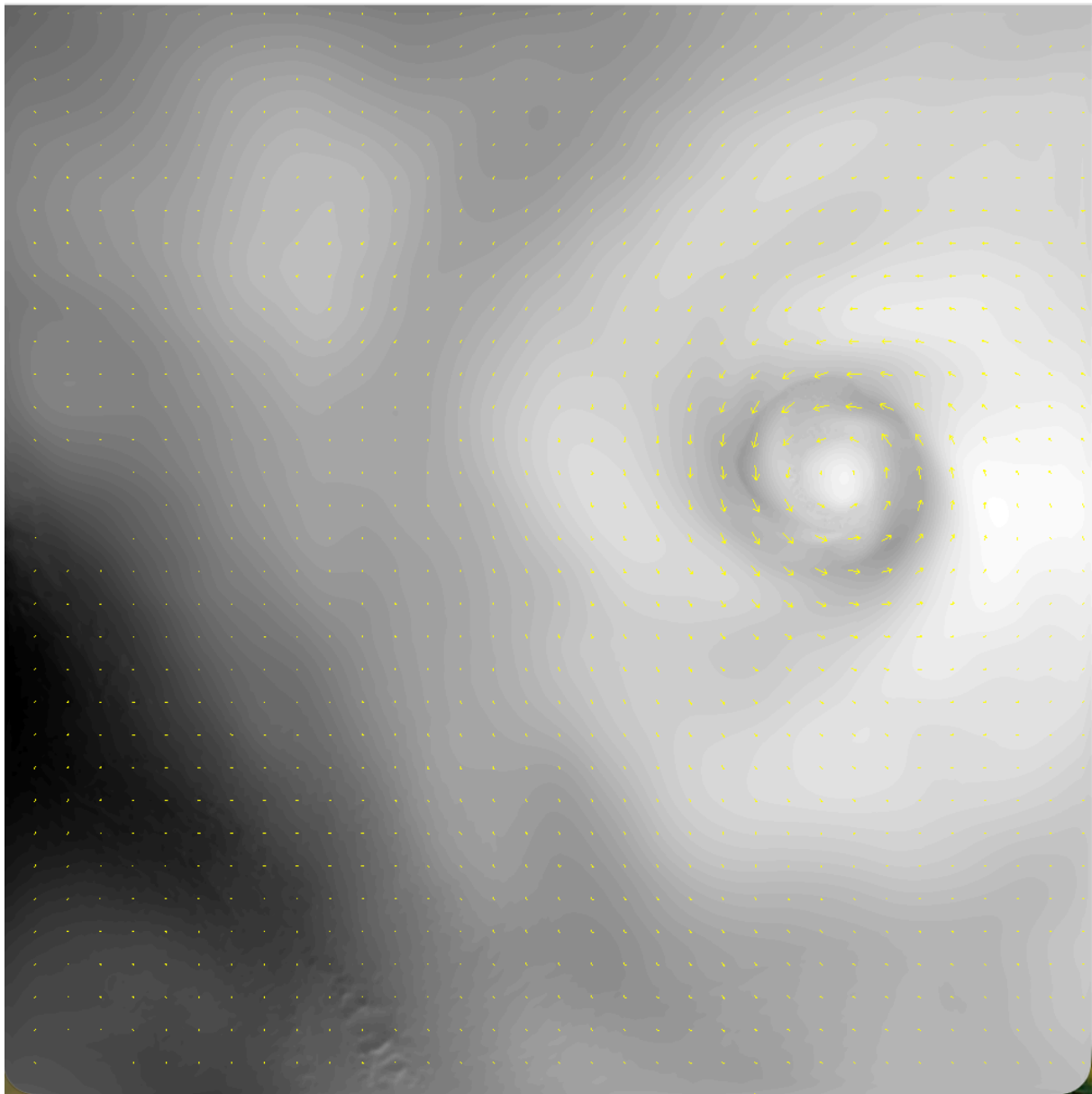
In the beginning I did not know how exactly the glyphs, streamlines and path lines were to be stored in the VBOs and VAOs.

I faced the problem with the stream lines. At first it did not get recomputed with the time step increment. Turns out we need to store the click position and the reintegrate every time the time step is incremented. A helper function(recomputeAllStreamlines) was implemented in order to achieve this result which recalculate all the integrals every time we increment the time step.

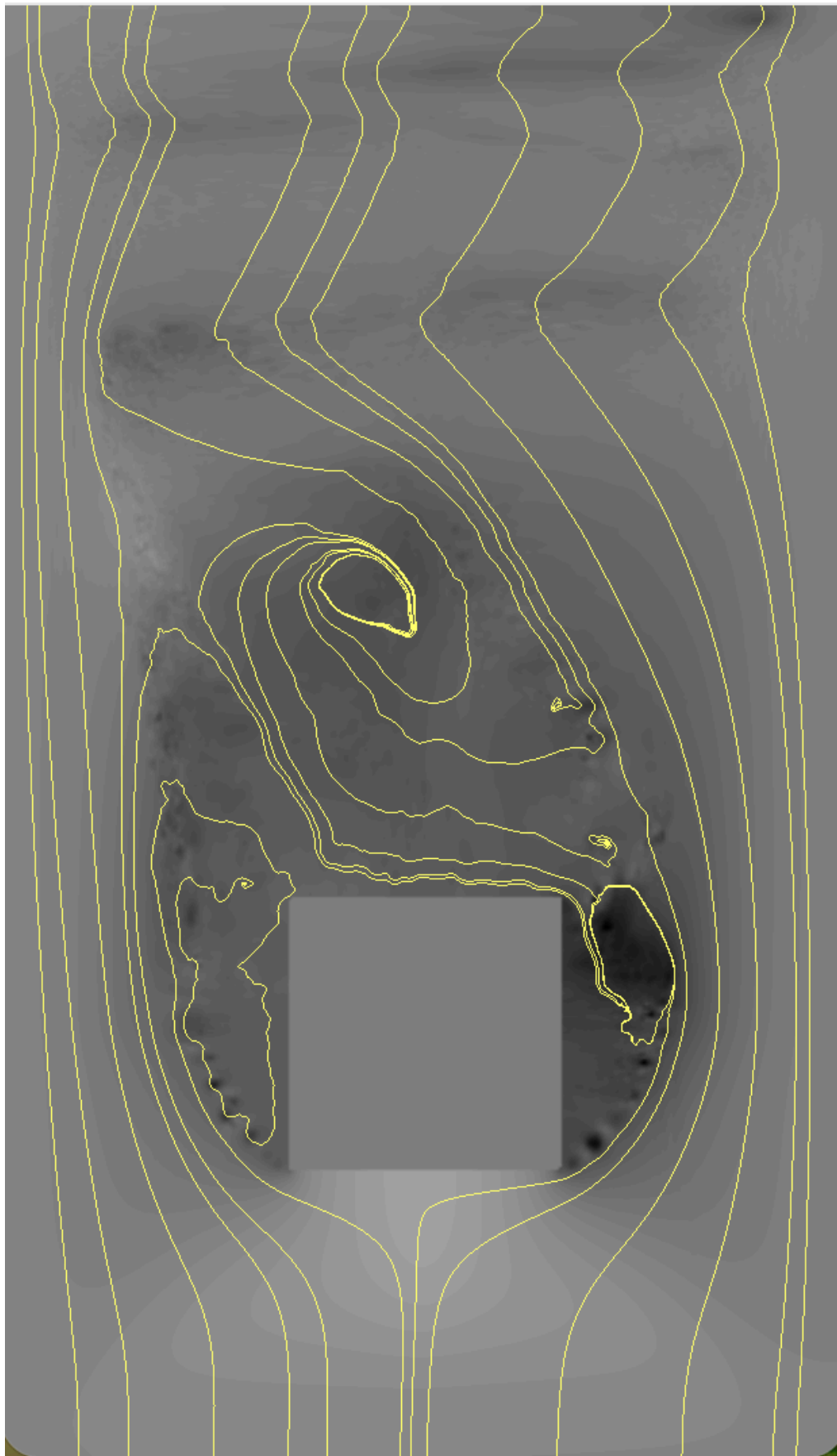
Demo:

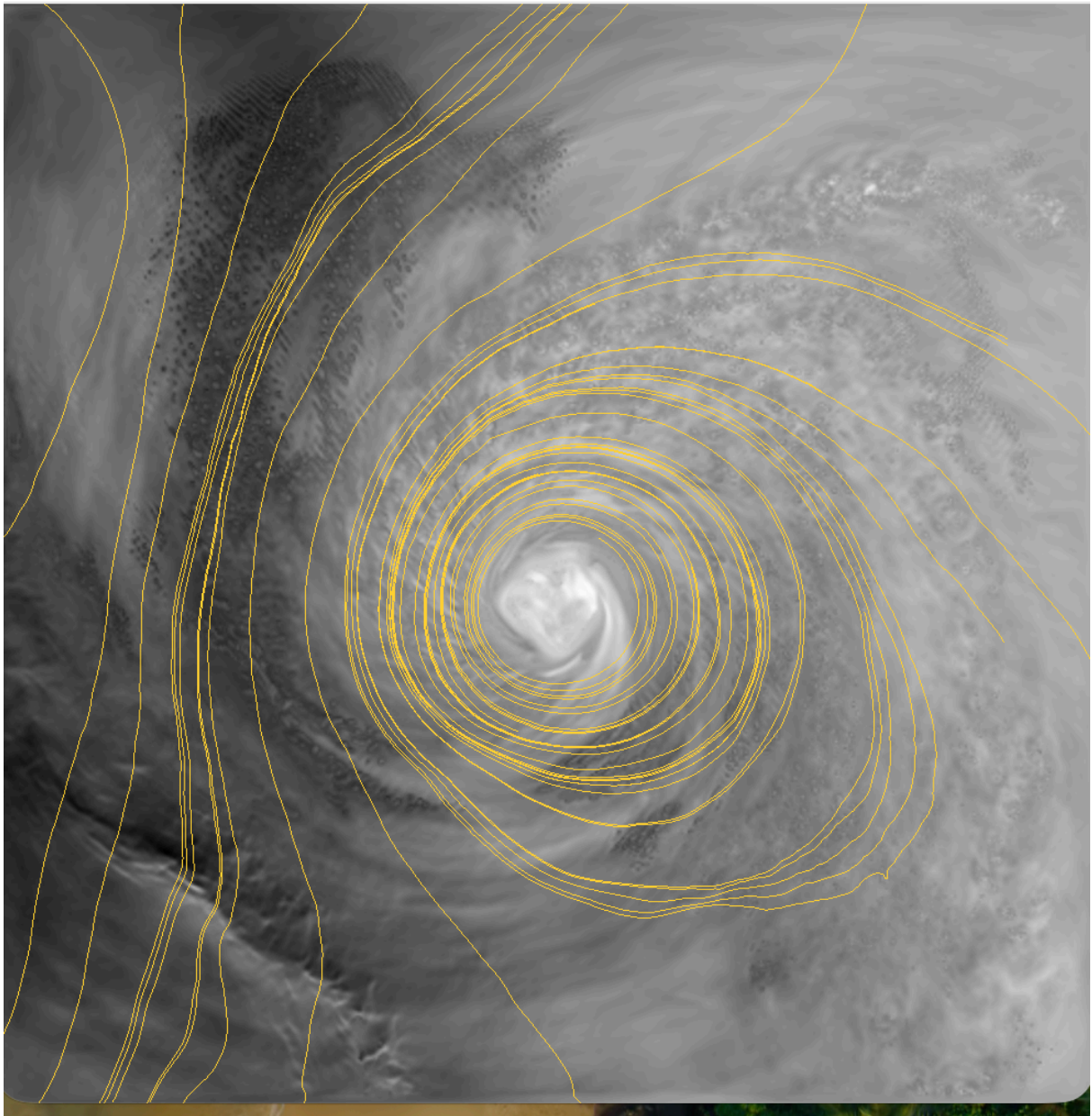
Glyphs Visualization:





Streamlines





Path lines + stream lines

